

Explaining Optimization Models Using Large Language Models

Can Li
Davidson School of Chemical Engineering
Purdue University

OR/CP Modeling Pipeline



Objective Function

Minimize total cost:

$$\min \sum_{t=1}^T (Ky_t + cx_t + hI_t + Bs_t)$$

Constraints

1. Inventory balance:

$$I_{t-1} + x_t - s_{t-1} = D_t + I_t - s_t \quad \forall t = 1, \dots, T$$

with I_0 and s_0 given.

2. Ordering constraint:

$$x_t \leq My_t \quad \forall t = 1, \dots, T$$

3. Non-negativity:

$$x_t, I_t, s_t \geq 0 \quad \forall t = 1, \dots, T$$

$$y_t \in \{0, 1\} \quad \forall t = 1, \dots, T$$

```
model.T = RangeSet(1, T)

# Decision Variables
model.x = Var(model.T, domain=NonNegativeReals) # Order quantity
model.y = Var(model.T, domain=Binary) # 1 if order placed, 0 otherwise
model.I = Var(model.T, domain=NonNegativeReals) # Inventory at end of period

# Objective Function
def total_cost(model):
    return sum(K * model.y[t] + c * model.x[t] + h * model.I[t] for t in model)
model.obj = Objective(rule=total_cost, sense=minimize)

# Constraints
def inventory_balance(model, t):
    if t == 1:
        return I0 * model.x[t] == demand[t] + model.I[t]
    else:
        return model.I[t-1] + model.x[t] == demand[t] + model.I[t]
model.inventory_balance = Constraint(model.T, rule=inventory_balance)

def ordering_logic(model, t):
    return model.x[t] <= M * model.y[t]
model.ordering_logic = Constraint(model.T, rule=ordering_logic)
```



Describe the problem (given .., determine .., under .. constraints)

Formulate as an optimization model

Solve with an optimization solver or algorithm.

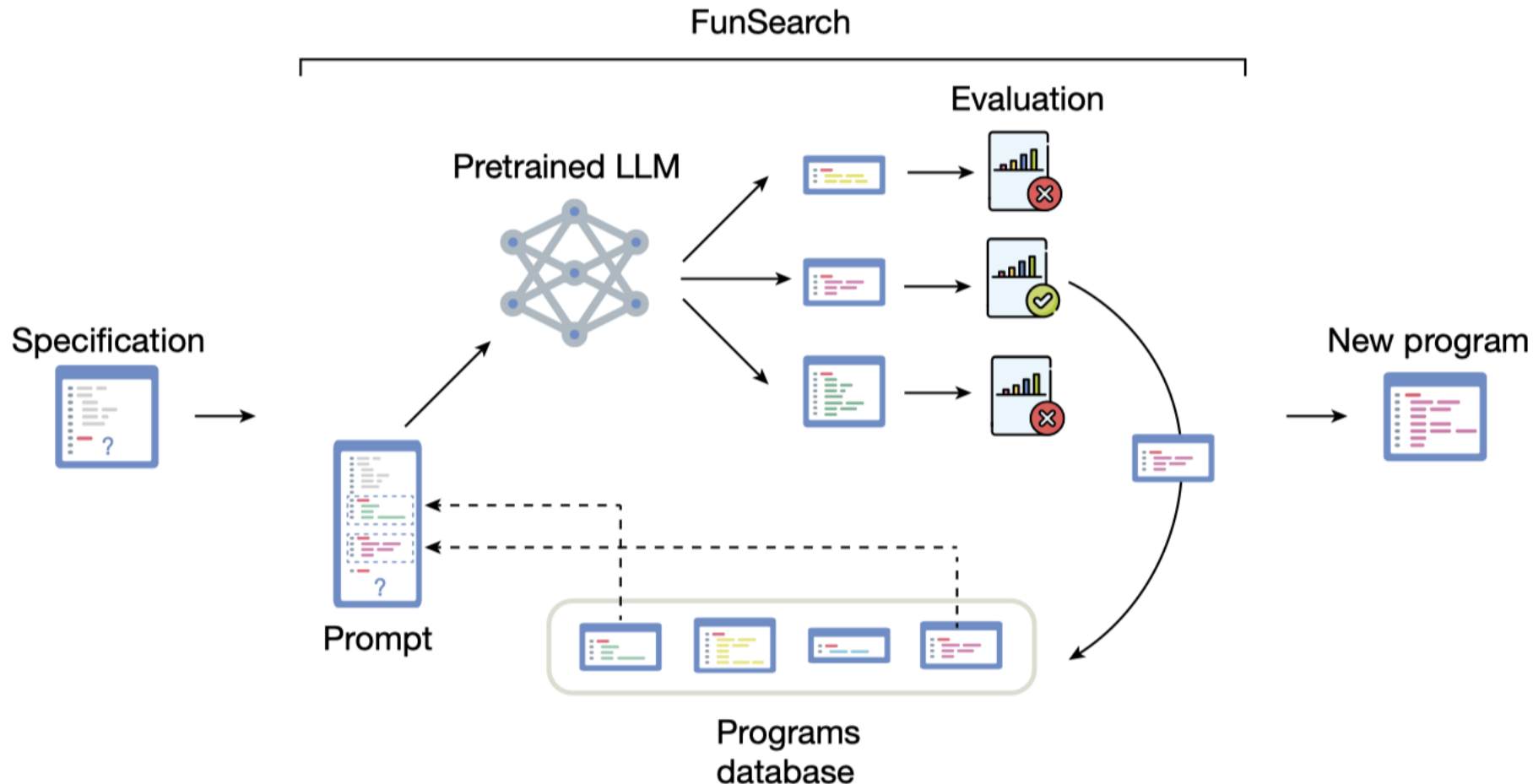
Interpret the optimal solution

OR/CP experts are involved at each step.

Can the pipeline be accelerated/improved using GenAI/LLMs?

- LLM assists modeling
 - OptiMUS: natural language to model (AhmadiTeshnizi et al., 2023)
 - Text2Model: Copilots for Text-to-Model Translation (Kadioğlu et al., 2026)
 - Cp-bench: LLM for constraint modelling (Michailidis, Tsouros, Guns, 2025)
 - CP-Model-Zoo: natural language to CP model (Crespin et al. 2025)
- LLM assists algorithm development and proofs
 - FunSearch/AlphaEvolve (Romera-Paredes et al., 2024)
 - HeuriGym: LLM for Combinatorial Optimization (Chen et al. 2026)
 - OptProver: a trained LLM for solving undergrad level optimization (Li et al. 2026)
- LLM facilitates optimization models and user interaction
 - OptiGuide: LLM for interpreting supply chain models (Li et al., 2023)
 - "I Want It That Way": Decision support via LLM and CP (Lawless et al., 2024)
 - **OptiChat**: Explain optimization models using LLM (Chen et al. 2023, 2025)

- Discover new optimization heuristics by using LLMs for generating new programs
- LLMs act a mutation and crossover operator in a genetic algorithm to search for better algorithms



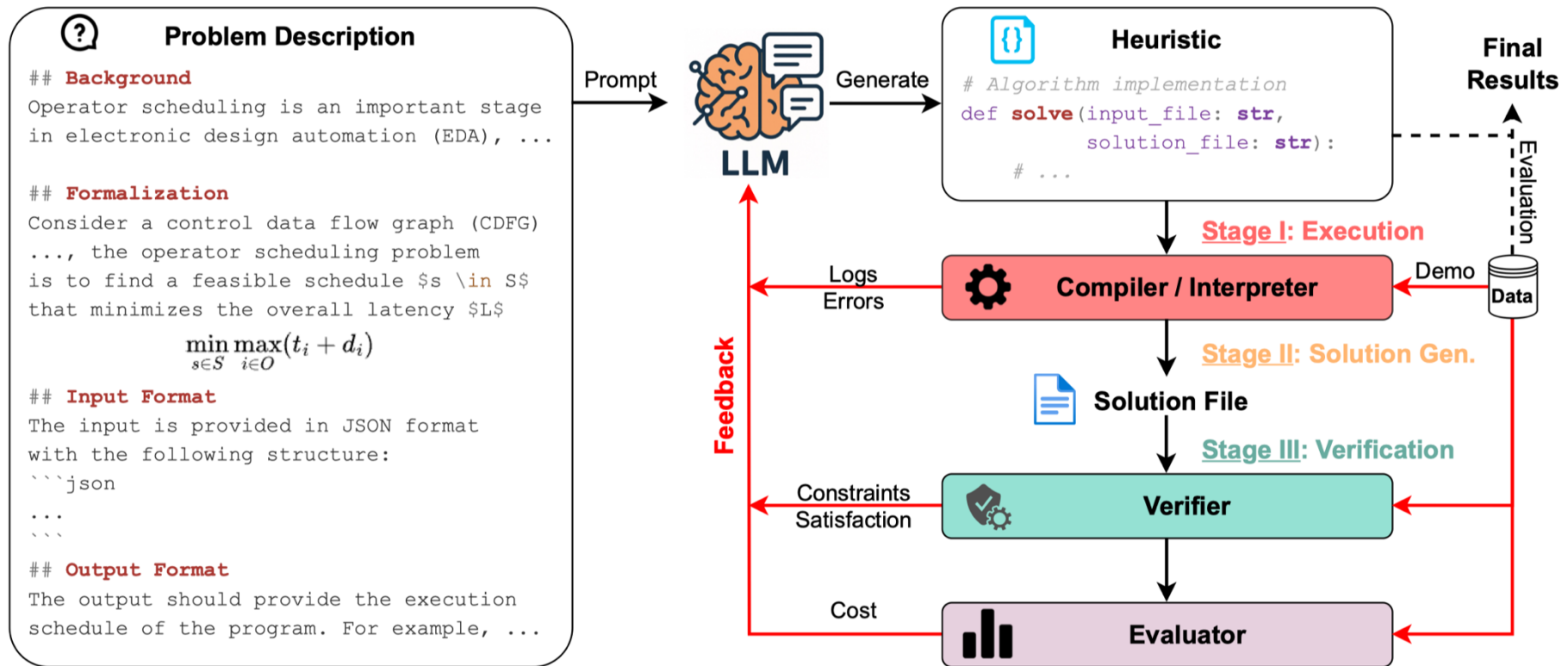
- Outperform best fit, first fit heuristics

Optimality gap w.r.t. offline bin packing lower bound

	OR1	OR2	OR3	OR4	Weibull 5k	Weibull 10k	Weibull 100k
First fit	6.42%	6.45%	5.74%	5.23%	4.23%	4.20%	4.00%
Best fit	5.81%	6.06%	5.37%	4.94%	3.98%	3.90%	3.79%
FunSearch	5.30%	4.19%	3.11%	2.47%	0.68%	0.32%	0.03%

```
def heuristic(item: float, bins: np.ndarray) -> np.ndarray:
    """Online bin packing heuristic discovered with FunSearch."""
    score = 1000 * np.ones(bins.shape)
    # Penalize bins with large capacities.
    score -= bins * (bins-item)
    # Extract index of bin with best fit.
    index = np.argmin(bins)
    # Scale score of best fit bin by item size.
    score[index] *= item
    # Penalize best fit bin if fit is not tight.
    score[index] -= (bins[index] - item)**4
    return score
```

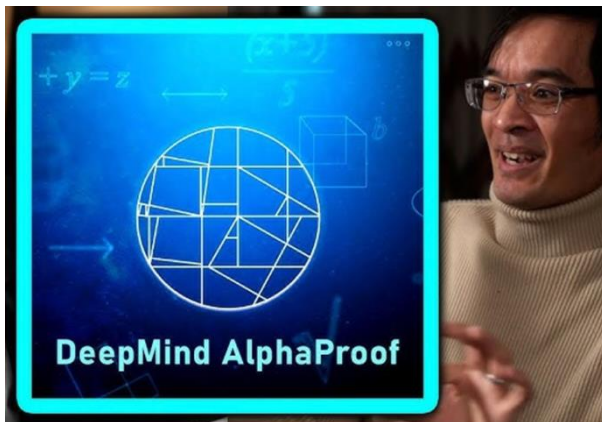
- Agentic benchmark for LLM-crafted heuristic for a library of combinatorial optimization problems including operator scheduling, technology mapping, global routing E-graph extraction, protein sequence design, airline crew pairing, etc.



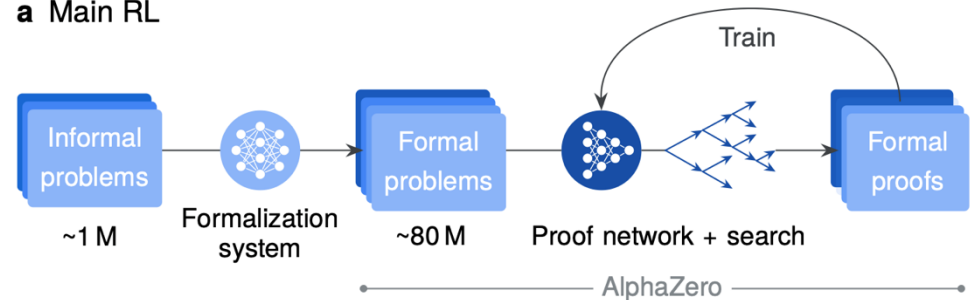
AI-assisted Theorem Proving



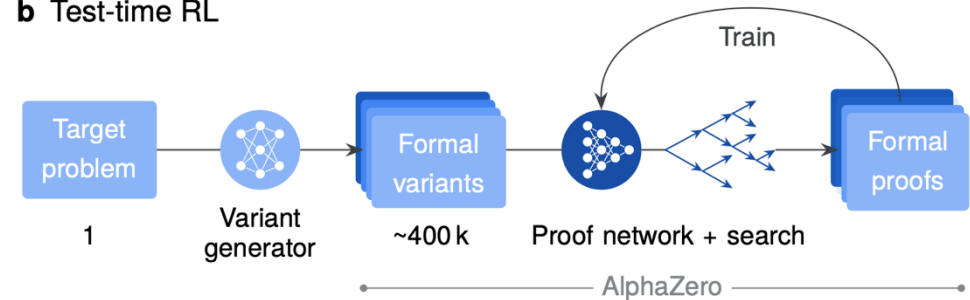
- OpenAI and DeepMind's generative AI tool achieve gold medal level performance at the International Math Olympiad (IMO) in 2025.
- British mathematician Timothy Gowers says OpenAI's ChatGPT 5.5 Pro solved open mathematics problems and produced doctoral-level research papers in less than two hours without mathematical guidance from him. (May 2026)



a Main RL

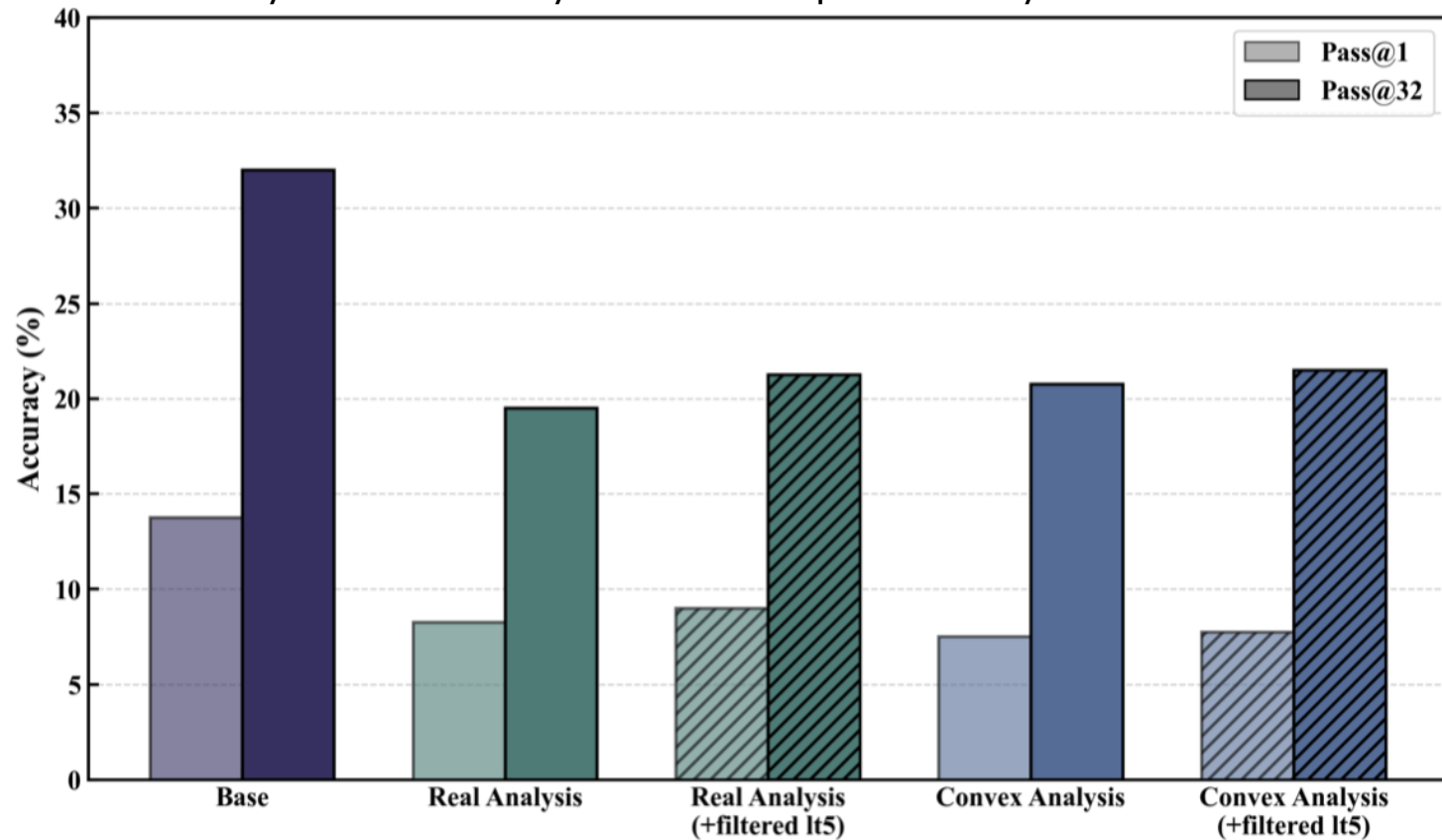


b Test-time RL



LEM
THEOREM PROVER

- Train an LLM for proving undergrad level optimization problems from the base model BFS-Prover-V2-7B (trained to solve Olympiad math problems).
- Direct supervised fine tuning on the whole proof cause “catastrophic forgetting”.
- The LLM learns syntactic validity rather than proof utility.



- OptProver uses a multi-stage continual training framework that addresses the challenges of distribution shift and catastrophic forgetting.
- **Expert-iteration curation:** Generate new proofs with the current prover, keep verified successes, and retrain.
- **Utility-aware preference learning:** Prefer proof-progressing tactics over valid but stagnant ones.
- **Perplexity-weighted DPO:** Down-weight out-of-distribution tokens to stabilize training and reduce forgetting.

Method	Basic		Convex		Algorithmic		Avg	
	Pass@1	Pass@32	Pass@1	Pass@32	Pass@1	Pass@32	Pass@1	Pass@32
Step-level Provers								
BFS-Prover-V2-7B	14.05%	36.36%	20.00%	44.44%	6.25%	16.67%	13.25%	32.00%
OptProver Variants (Ours)								
OptProver (Only EI)	32.23%	49.59%	39.26%	60.00%	15.97%	40.97%	28.75%	50.00%
OptProver (EI + DPO)	34.71%	52.07%	39.26%	62.22%	20.14%	42.36%	31.00%	52.00%
OptProver (EI + UAPO)	36.36%	53.72%	45.93%	60.00%	27.08%	47.22%	36.25%	53.50%
OptProver (EI + PW-DPO)	37.19%	53.72%	45.19%	62.96%	27.08%	46.53%	36.25%	54.25%
OptProver (EI + PW-UPAO)	40.50%	55.37%	45.19%	62.22%	28.47%	48.61%	37.75%	55.25%

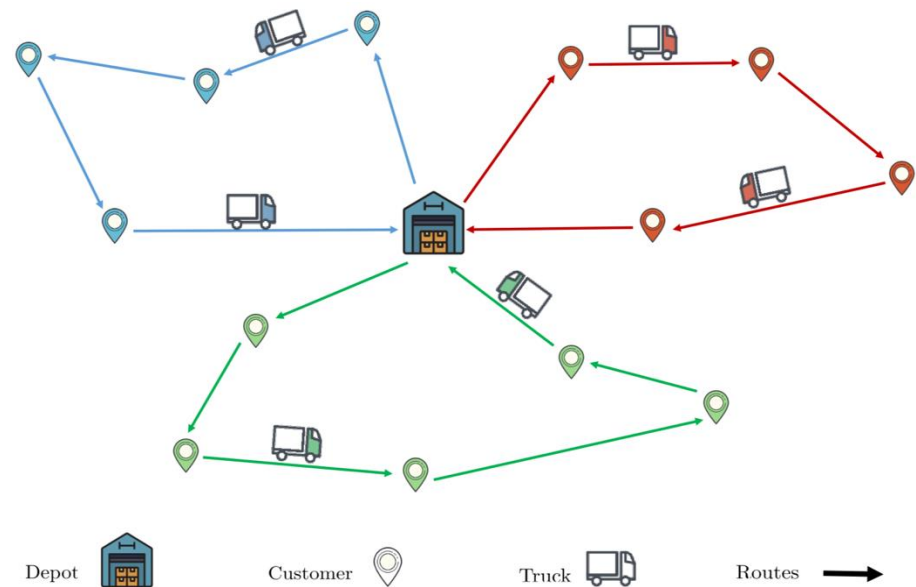
- Optimization models are typically developed by OR experts but are used by practitioners without any expertise in optimization.
- Major barrier of using optimization models in practice: **lack of interpretability**

Interpret infeasible models



Southwest	277
Cancelled	
Southwest	182
Cancelled	
Southwest	315
Cancelled	
Southwest	337
Cancelled	
Southwest	132
Cancelled	

Interpret why a certain route is optimal



- Active area since 1980s
- Expert system (Greenberg, 1983)
- Infeasibility detection (Chinneck, 1991, Puranik and Sahinidis, 2017)
- Interpretable decision rules (Bertsimas and Stellato, 2021; Lumbreras et al., 2024)
- Argumentation-based methods (Cyras et al., 2019; Collins et al., 2019)
- Counterfactual explanations (Forel et al., 2023)
- Fuzzy inference (Deneke et al., 2024)
- Explain stochastic programming models (Rathi et al. 2024)

Although experts can gain insights from expert systems and explanation techniques, practitioners still struggle to get useful information independently

- Not a new problem.
- Greenberg and coworkers developed the ANALYZE system in the 1980s.

```

1 ANALYZE
2 EXPLAIN ROWS?
3 Rows that begin with W=min amount shipped for some timber merchant
4 M=min amount produced for some wood plant
5 X=max amount produced for some wood plant
6 C=min customer demand for some chair retailer
7 T=max customer demand for some chair retailer
8 B=stock for some wood plant
9 ANALYZE
10 * check total supply versus total demand
11 ANALYZE
12 ADDRIM ROW * L=0/*
13 Number      Level      PR      LO Bound  UP Bound  Dual Price
14 -----
15 14          34500.0    0      36250.0    *        - 3.00
16 ANALYZE
17 ADDRIM ROW * U=*/99999
18 Number      Level      PR      LO Bound  UP Bound  Dual Price
19 -----
20 12          5000.0    0      - *        7900.0    - 20.00
21 ANALYZE
22 * demand exceeds supply and is out of kilter
23 ANALYZE
24 DISPLAY ROW * L=7900/*
25 Name      Stat  Level      PR      LO Bound  UP Bound  Dual Price
26 -----
27 WMON      I    13500.0    0      16000.0    *        0
28 WMQU      L    16000.0    0      16000.0    *        2.0
29 ANALYZE
30 EXPLAIN
31 Row WMON demands min amount shipped for Ontario
32 Row WMQU demands min amount shipped for Quebec
33 ANALYZE
34 * the min demands for Ontario and Quebec are out of kilter!
    
```

Ideally, one would like to move toward a natural language query system where the descriptions are resident with the model in a way that permits more automation to obtain answers.

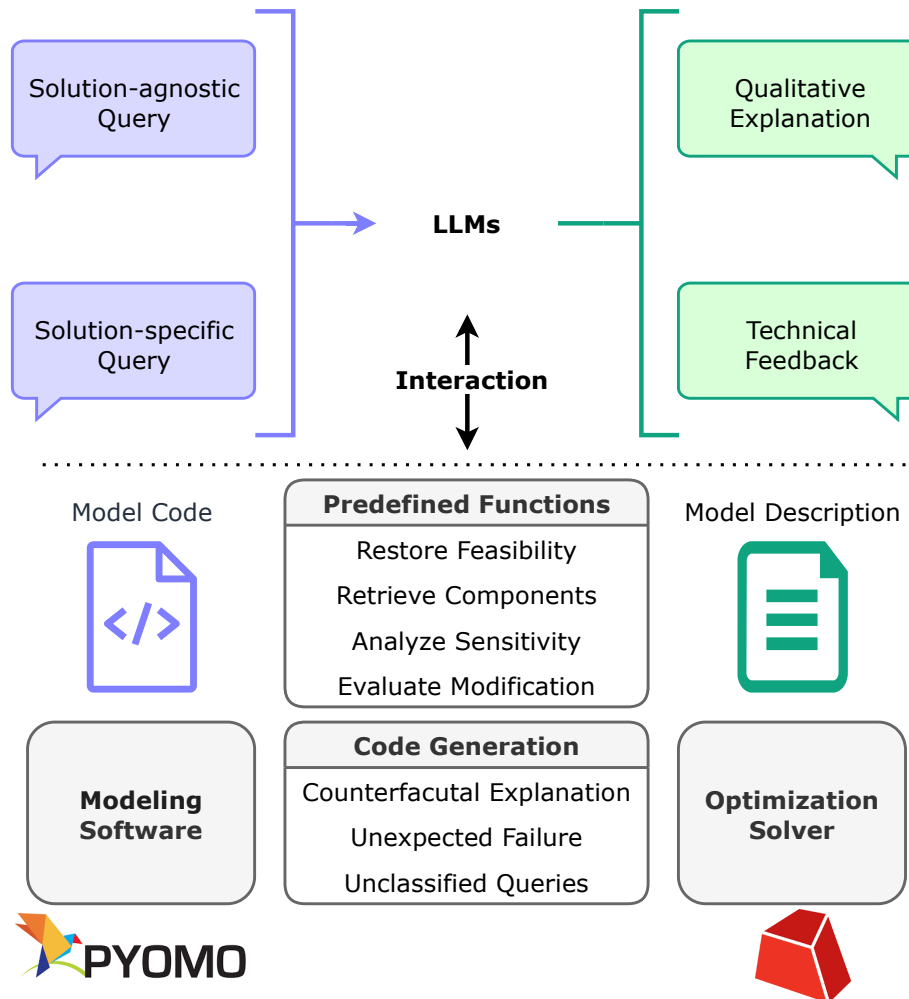
(Greenberg 1983)



- Hallucination
- Large optimization models
- What explanation techniques to use?

- OptiChat takes a Pyomo model written by an optimization expert and answers the following questions.
 - Solution-agnostic (LLM answers from model structure alone)
 - Problem description & explanation
 - Retrieval (Parameters)
 - Solution-specific (Requires solving and/or modifying the model)
 - Retrieval (Solution/ Dual)
 - Infeasibility diagnosis & restoration
 - Sensitivity analysis/ What-if
 - Contrastive (Why-not)
 - Robustness analysis
 - Linear decision rules
 - Rolling horizon queries

- An agentic framework where the LLM interacts with function calls and code generator, and outputs trustworthy technical feedback.



Query

Retrieve specific values or components from the solved model

➤ Examples

- "What is the optimal production quantity at facility 2?"
- "Which suppliers are selected in the optimal solution?"
- "What is the dual price of the demand constraint at period $t=3$?"

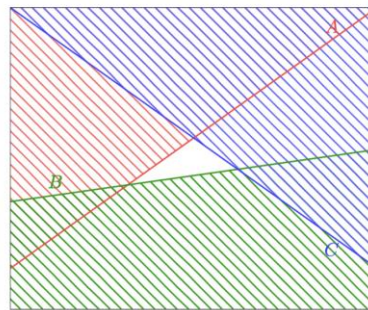
➤ How it works

- LLM calls predefined retrieval function & returns value and explanation in plain language

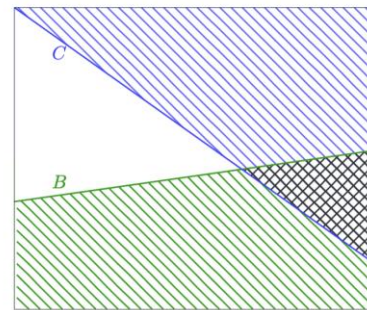
➤ Supported retrievals

- Variable values
- Objective value
- Parameter values
- Constraint slack
- Dual variables and reduced costs

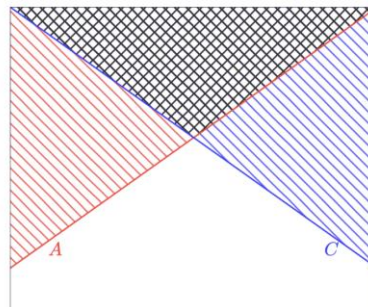
- **Irreducible Infeasible Subset (IIS)** (Chinneck and Dravnieks 1991): a minimal subset of constraints and/or variable bounds within an optimization model that is infeasible.
- Algorithms to find IIS
 - **Deletion filter:** systematically remove constraints and check for feasibility.



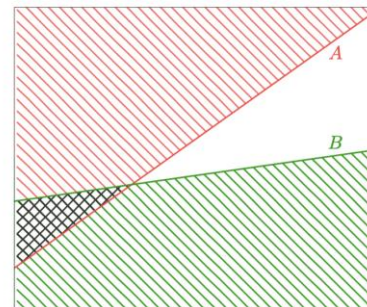
(a) Original constraint set



(b) Remove constraint A



(c) Remove constraint B



(d) Remove constraint C

Solvers like Gurobi provides IIS detector implementations

- Add minimal slacks to the parameters to resolve the infeasibility.
- Adding δA^+ and δA^- makes the problem nonlinear and should be discouraged by the chatbot

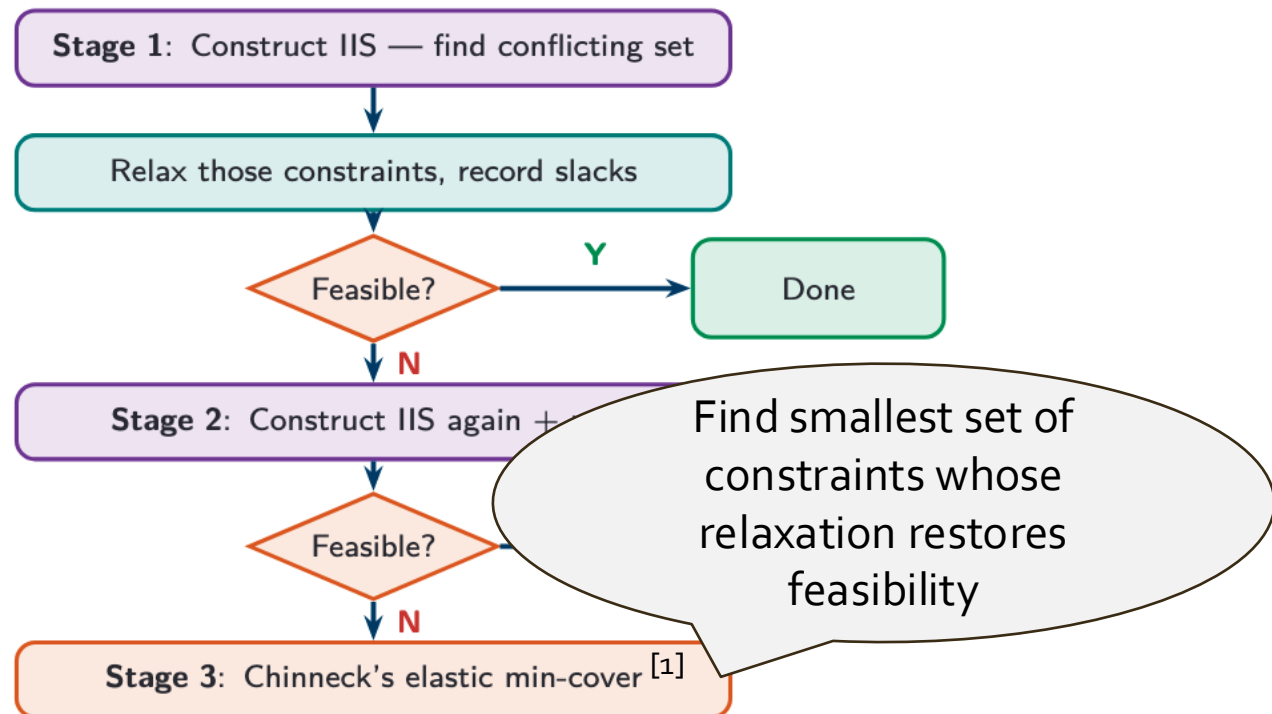
$$\begin{aligned} \min_{\mathbf{x}, \delta \mathbf{A}^+, \delta \mathbf{A}^-, \delta \mathbf{b}^+} \quad & \sum_{(i,j) \in \mathcal{S}_A} (\delta \mathbf{A}_{ij}^+ + \delta \mathbf{A}_{ij}^-) + \sum_{i \in \mathcal{S}_b} \delta \mathbf{b}_i^+ \\ \text{subject to} \quad & (\mathbf{A} + \delta \mathbf{A}^+ - \delta \mathbf{A}^-) \mathbf{x} \leq \mathbf{b} + \delta \mathbf{b}^+, \\ & \mathbf{x} \in \mathbb{Z}^p \times \mathbb{R}^{n-p}, \quad \delta \mathbf{A}^+, \delta \mathbf{A}^-, \delta \mathbf{b}^+ \geq 0. \end{aligned}$$

A function is implemented in OptiChat for resolving infeasibility.

Limitation of single IIS call

IIS returns one conflict. A model may be infeasible due to several independent conflicts

Iterative IIS + Min Cover Strategy



Query

Understand how variations in input parameters affect the optimal solution and objective value

➤ Examples

- How does changing the production capacity affect the profit margin?
- How much will the cost increase if the fixed cost of supplier A is increased to \$10?

➤ Sensitivity analysis is completed by two predefined functions

- **Sensitivity Query:** RHS parameters and LP models: get the dual variable of the constraint
- **What-if Query:** Modify the constraint and resolve

Query

Explain why the user recommended solution is not optimal

➤ Example

- “Why the solution is not A?”

➤ Method

- Add a forcing constraint to make A happen (e.g., $y_1 = 1$)
- Re-solve the model
- Report feasibility and objective change — cost increase / profit decrease

Requires code generation, LLM writes and executes the forcing constraint in Pyomo and resolves the model



RUNNING...

Stop

Deploy

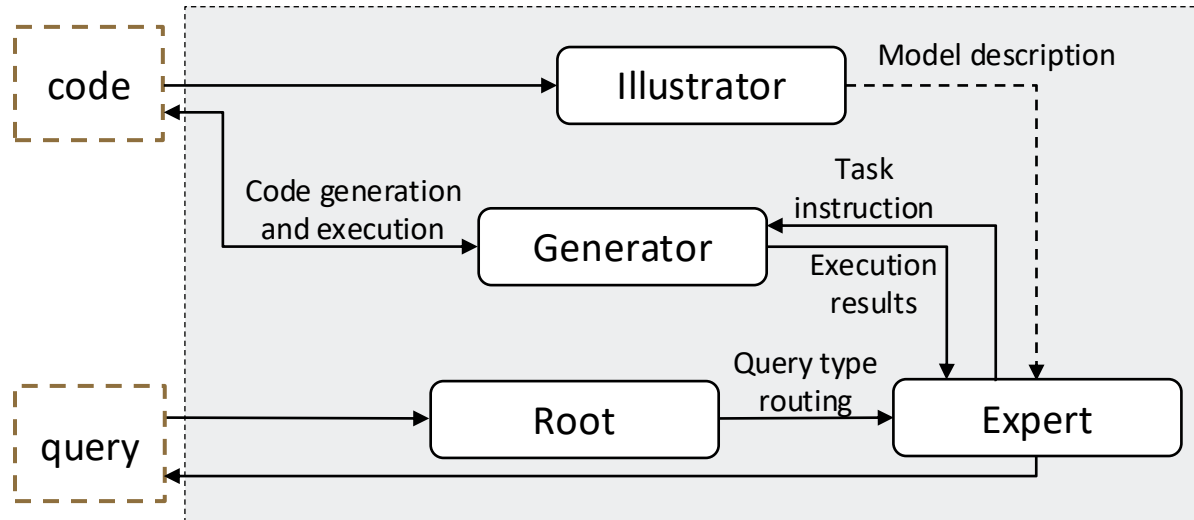


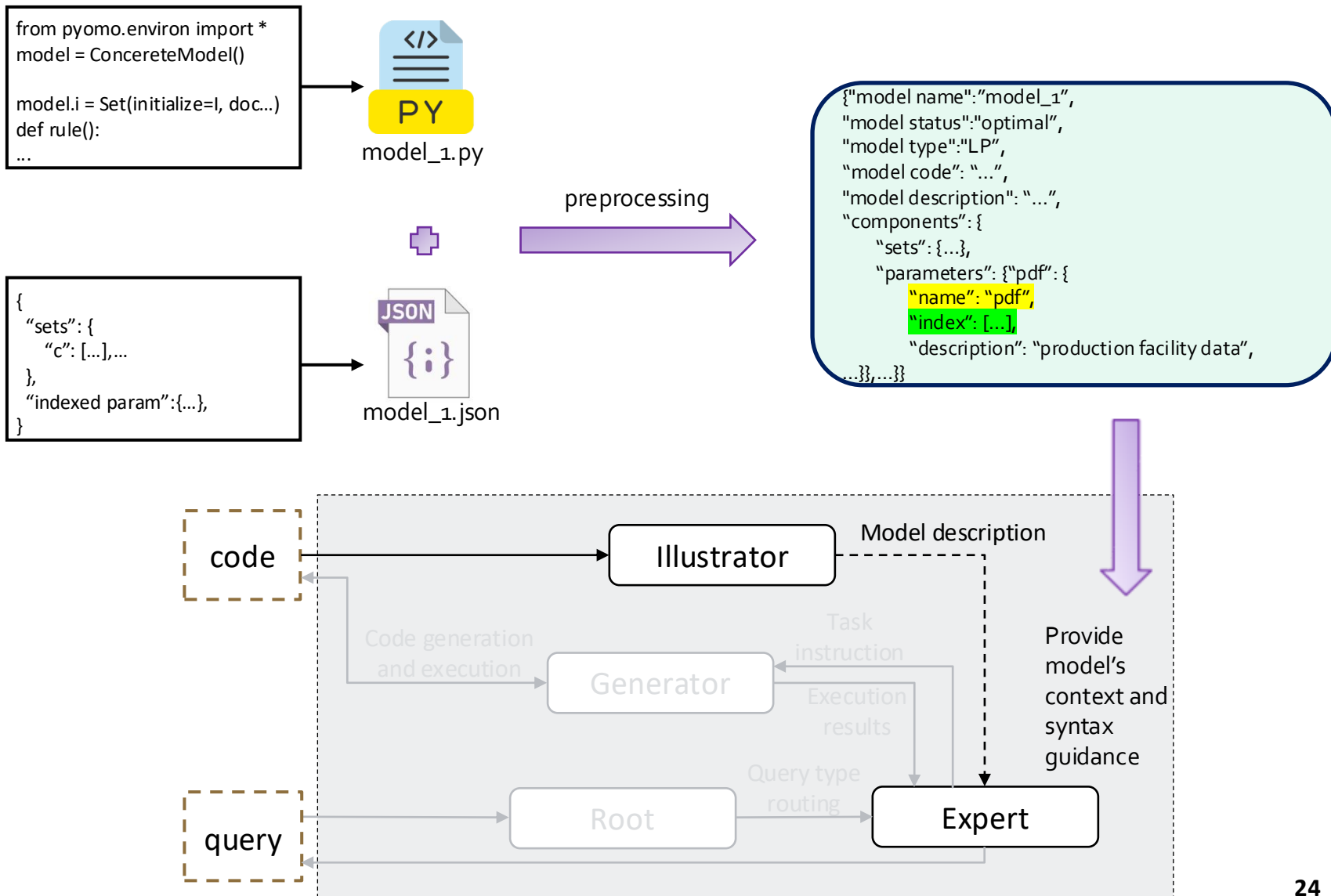
I have uploaded a Pyomo model.

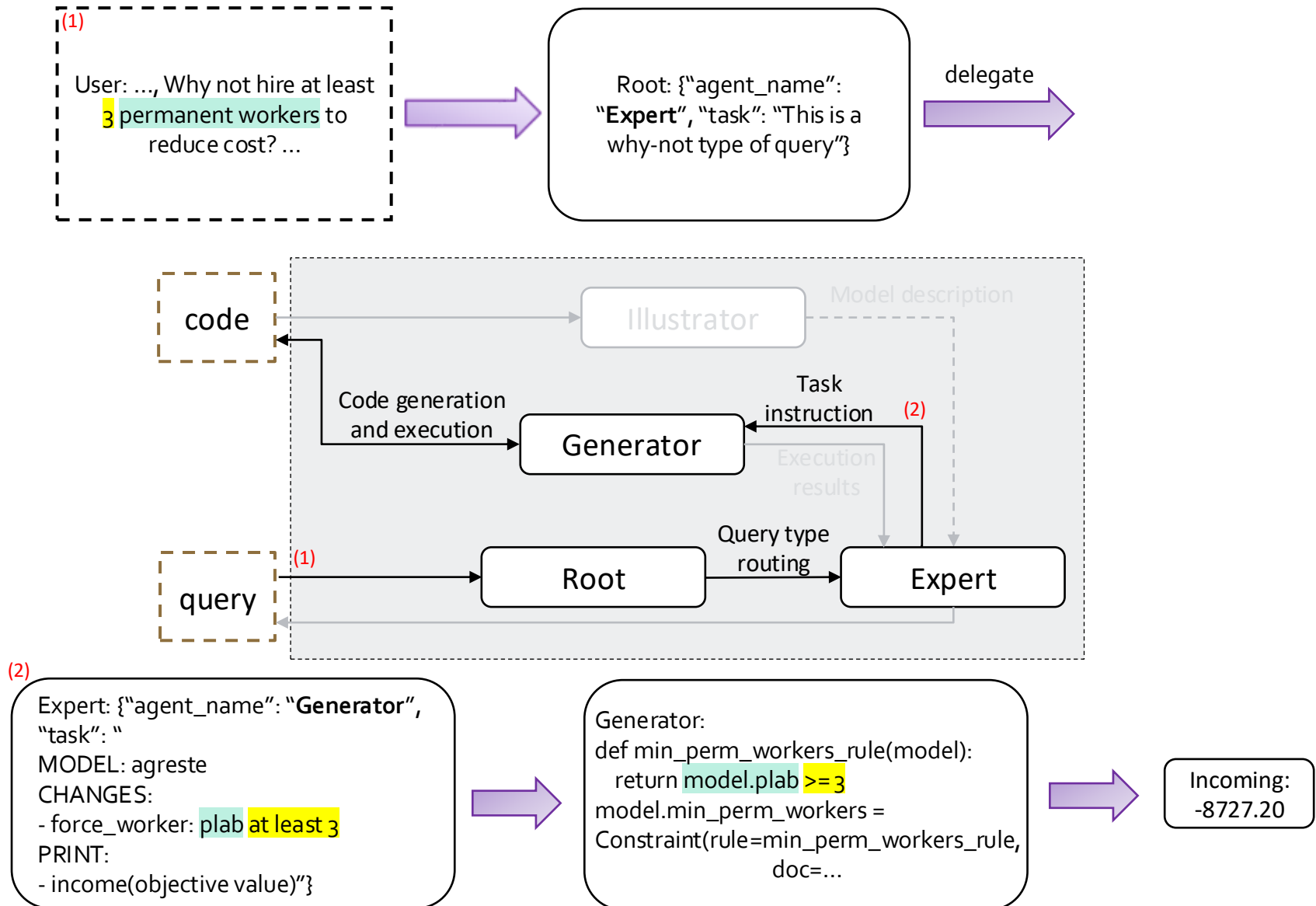
Enter your query here...

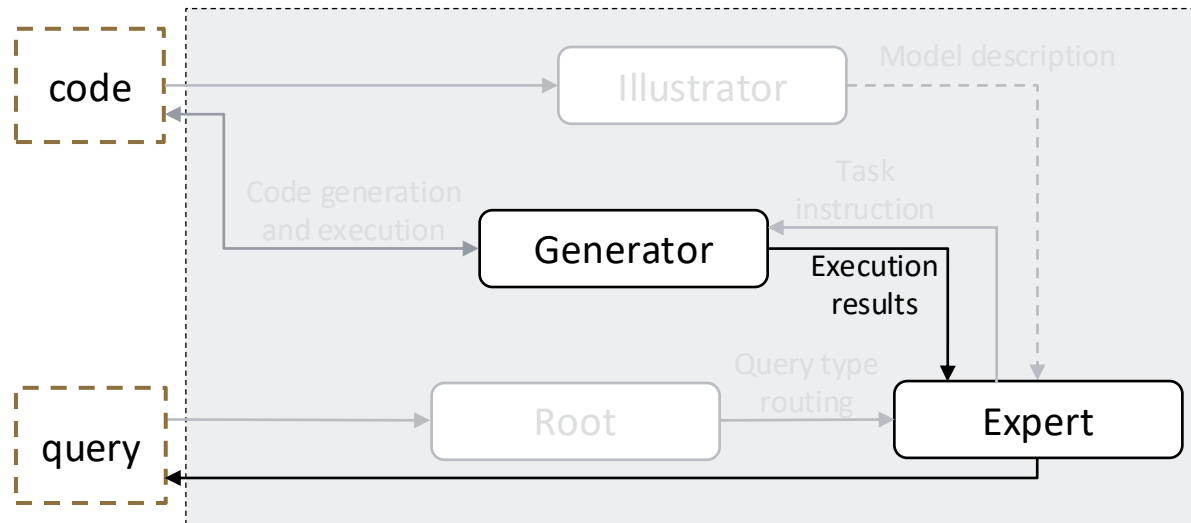


- **Illustrator:** extracts sets, parameters, variables, constraints, and objectives from the Pyomo optimization model. Establish a dictionary that maps the meanings of model components to their notations in the source code.
- **Root:** identify the type of query and assign corresponding workflow to Expert.
- **Expert:** decide what function should be used or generate instruction to guide Generator.
- **Generator:** generate and execute codes to get the desired results back to Expert.









Generator: {"agent_name": "**Expert**",
"task": "The new objective value is:
-8727.20, which is 25004.69 lower
than previous objective value
(16277.49)"}



Expert: The optimal solution
suggests using 2 permanent
works only. If using more
than 3 permanent workers...

- Benchmark dataset with 24 optimization models and consists of 172 QA pairs
- Model sources: GAMS models library, ND-Pyomo Cookbook, Optimization in Operations Research textbook by Ronald L. Rardin
- Infeasible variants of the 24 models are created by adjusting model parameters or introducing additional constraints

Category	Count
# Total Models	24
# LP	10
# MILP	14
# Parameters (Min./Avg./Max.)	21 / 1,264 / 25,432
# Variables (Min./Avg./Max.)	14 / 1,002 / 14,880
# Constraints (Min./Avg./Max.)	10 / 321 / 2,672
Manufacturing & Scheduling	8
Supply Chain & Logistics	8
Chemical & Process Engineering	4
Power & Energy	3
Other	1

Accurate and Trustworthy Explanations

- We categorize practitioners' questions into (1) feasibility restoration, (2) retrieval, (3) sensitivity, (4) what-if, and (5) why-not queries
- Queries can be systematically identified and handled by OptiChat
- Our predefined functions and tailored code generation result in ~90% success rates
- The new OptiChat outperforms the performance with legacy framework

Query	Old				New			
	Accuracy (%)		Time (min)		Accuracy (%)		Time (min)	
	GPT-4o	o3	GPT-4o	o3	GPT-4o	GPT-5	GPT-4o	GPT-5
Feasibility restoration	84.6	89.7	0.2	0.6	97.4	100	0.4	1.0
Retrieval	92.3	97.4	0.1	0.3	97.4	100	0.1	0.1
Sensitivity	94.4	94.4	0.2	0.5	82.4	100	0.2	0.3
What-if	94.9	87.2	0.2	0.4	85.7	96	0.3	0.4
Why-not	54.1	78.4	0.4	0.9	65.4	80.8	0.3	0.7
Overall	83.1	88.9	0.2	0.5	86.6	95.9	0.3	0.5

- Syntax Error: code execution failures and invalid function calls
- Classification: the explanation strategy chosen by the LLM does not match the human-annotated query type, e.g., a why-not query being misclassified as a what-if query
- Logic Error: (1) The LLM fails to represent a counterfactual scenario using constraints during code generation. (2) Component names or component indexes produced by the LLM exist in the model as valid function arguments but do not accurately reflect the user's intent.

Error Type	GPT-4.1 (%)	o3 (%)
Syntax Error	46.8	10.0
Classification Error	31.3	25.0
Logic Error	21.9	65.0

Query

How much can a **parameter** change before the current solution x^* becomes infeasible?

➤ Examples

- "How much can demand grow before our current plan breaks?"
- "By how much can machine capacity fall while this schedule holds?"

➤ Method (Per-constraint margin)

For constraint $a_i^\top x^* \leq b_i$

$$\delta_i^* = b_i - a_i^\top x^* \geq 0$$

$\delta_i^* = 0 \Rightarrow$ Binding

b_i decrease by at most δ_i^*

If a parameter appears in multiple constraints, the allowable change is the minimum slack across all constraints containing it.

Query

Which parameters drive changes in the optimal decisions, and by how much?

➤ Linear Decision Rule (LDR):

$$\mathbf{x}^*(\xi) = \mathbf{a} + B \xi$$

- $\mathbf{a} \in \mathbb{R}^m$: Intercept
- $B \in \mathbb{R}^{m \times m}$: Coefficients
- $\xi \in \mathbb{R}^m$: Parameters (Uncertain)
- Parameter space: $W\xi \geq h$
- Row 'i' of B: how decision x_i responds to parameters ξ

➤ Examples

- "How does order quantity depend on demand?"
- "Which parameter most influences facility 2?"

Advantage

Answer what-if questions without re-solving the model

➤ Examples

- "If demand at facility 2 rises to 1,200 units, what is the order quantity?"
- "How does the production quantity change at facility 1 if cost to produce increases by 10%?"

➤ Substitute $\tilde{\xi}$ into the LDR

$$\mathbf{x}^*(\tilde{\xi}) = \mathbf{a} + B \tilde{\xi}$$

- Valid as long as $\tilde{\xi}$ is in the parameter space $W\xi \geq h$ used to fit the LDR
- For $\tilde{\xi}$ outside the parameter space, LLM falls back to regular what-if by resolving the model

Advantage

Given a desired range of decisions, find parameters ξ that achieves it

➤ Examples

- "What demand levels keep the order quantity between 400 and 600 units?"
 - "How much should the initial inventory be so that my reorder quantity does not exceed 20?"
- $\mathbf{a}$, $\mathbf{B}$, ξ^0 are all known. ξ is the only variable: Project ξ^0 onto a feasible parameter set

$$\min_{\xi} |\xi - \xi^0| \quad \text{s.t.} \quad \mathbf{lb} \leq \mathbf{a} + \mathbf{B}\xi \leq \mathbf{ub}$$

- L2 norm $\Rightarrow$ QP, L1/L ∞ $\Rightarrow$ LP
- LLM formulates and solves the projection problem

Setting

The model is re-solved at each period t as new information arrives. The optimal solution evolves, and practitioner asks: why?

Structural &
input changes

Solution
differences

Backward
compatibility

Query

What changed in the model between t and $t+1$ solves?

➤ Examples

- "Were any new routes added to the network this period?"
- "Did demand forecasts or capacity limits change?"
- RHS changes (forecasts, capacities)
- Constraints / variables added or removed
- Objective coefficient changes (costs, pricing)

Query

What are the key differences between x_t^* and x_{t+1}^* ?

➤ Examples

- "What changed in the production schedule from period t to $t+1$?"
- "Which suppliers were switched?"

➤ Binary decision

Activations / deactivations: $y_i : 0 \rightarrow 1$ or $1 \rightarrow 0$

➤ Continuous decision

Change greater than a threshold: $|x_{t+1,i}^* - x_{t,i}^*| > \varepsilon$

Query

Can the previous solution x_t^* still be executed in the updated model at period $t + 1$?

➤ Examples

- Can I use my previous scheduling plan?

➤ Method

- Overlapping variables fixed to x_t^* ; non-overlapping variables free; model at $t + 1$ resolved
- If feasible $\Rightarrow$ previous plan compatible
Updated plan constructed from re-solve optimality loss reported:
 $\Delta z = z_{t+1}^* - z_{t+1}^*$ (fixed)
- If infeasible $\Rightarrow$ previous plan incompatible
Infeasibility diagnosis (IIS + min-cover)
Feasibility restoration — minimum RHS relaxations

➤ Papers and preprints:

- Chen, H., Constante-Flores, G. E., & Li, C. (2024). Diagnosing infeasible optimization problems using large language models. *INFOR: Information Systems and Operational Research*, 1-15.
- Chen, H., Constante-Flores, G. E., Mantri, K. S. I., Kompalli, S. M., Ahluwalia, A. S., & Li, C. (2025). OptiChat: Bridging Optimization Models and Practitioners with Large Language Models. *INFORMS Journal on Data Science*

➤ Code:

- <https://github.com/li-group/OptiChat>

ORACLE

amazon